

Energy-Efficient Web Design: Measuring Impact of Front-end Optimizations

Jerin Anan Proma

Department of Computer Science and
Engineering
East West University
Dhaka, Bangladesh
jarinanaproma@gmail.com

Fatema Tuz Zannat

Department of Computer Science and
Engineering
East West University
Dhaka, Bangladesh
fatematuzzannat135@gmail.com

Mohammed Shawqi Aftab

Department of Computer Science and
Engineering
East West University
Dhaka, Bangladesh
shawqiaftab@gmail.com

Tahura Akter Tripty

Department of Computer Science and
Engineering
East West University
Dhaka, Bangladesh
triptyhossain977@gmail.com

M. Saddam Hossain Khan

Department of Computer Science and
Engineering
East West University
Dhaka, Bangladesh
saddam.cse@ewubd.edu

Raihan Ul Islam

Department of Computer Science and
Engineering
East West University
Dhaka, Bangladesh
raihan.islam@ewubd.edu

Rashedul Amin Tuhin

Department of Computer Science and
Engineering
East West University
Dhaka, Bangladesh
mccutuhin@ewubd.edu

Abstract

Today's websites include multimedia and interactive features that increase energy use and CO₂ emissions. Although front-end optimization techniques are common, their actual environmental impact has been underexplored. This study examined the effects of image compression, lazy loading, and code minification—individually and in combination—on client-side energy consumption and emissions. A controlled laboratory experiment was conducted with eight variants of the website, including a baseline and combinations of the three techniques. Energy use was measured with Firefox Energy Profiler for CPU and graphics processing components and with a Tuya smart plug for whole-device consumption. Automated browsing ensured consistent interactions across trials. The findings show that image compression provided the strongest single benefit, reducing system-level energy by 47.5%. The combinations achieved far greater effects, with image compression plus code minification cutting the energy of the entire device by 89.8% and CO₂ emissions by 89.6%. The results demonstrate that standard front-end optimizations, when applied together, substantially lower the environmental footprint of web applications.

CCS Concepts

• **Software and its engineering** → **Software performance**; *Empirical software validation*; • **General and reference** → **Empirical studies**; *Measurement*; *Evaluation*.

Keywords

Energy-efficient web design, Front-end optimization, Image compression, Lazy loading, Code minification, Sustainable web development, Client-side energy consumption, Web performance optimization

ACM Reference Format:

Jerin Anan Proma, Fatema Tuz Zannat, Mohammed Shawqi Aftab, Tahura Akter Tripty, M. Saddam Hossain Khan, Raihan Ul Islam, and Rashedul Amin Tuhin. 2025. Energy-Efficient Web Design: Measuring Impact of Front-end Optimizations. In *12th International Conference on Next Generation Computing, Communication, Systems and Security (NSysS '25)*, December 18–20, 2025, Sylhet, Bangladesh. ACM, New York, NY, USA, 9 pages. <https://doi.org/10.1145/3777555.3777561>

1 Introduction

One of the major supports of modern society, the Internet is now almost impossible to imagine living without. It has become the primary medium for many activities that once required physical presence or traditional methods, including work, education, commerce, entertainment, healthcare, and public services. As usage and dependence increase, the modern digital environment has grown into a vast network of interconnected websites. These websites have evolved from basic static documents into sophisticated platforms capable of delivering advanced and highly interactive user experiences. Contemporary web-based systems commonly incorporate



This work is licensed under a Creative Commons Attribution 4.0 International License. *NSysS '25, Sylhet, Bangladesh*
© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2122-9/25/12
<https://doi.org/10.1145/3777555.3777561>

multimedia content, adaptable layouts, client-side scripts, real-time communication, and personalized services. Although these capabilities improve accessibility and usability, they also demand greater processing power from the end devices and the supporting infrastructure.

Each interaction with a website requires the consumption of electrical energy across the entire digital delivery pipeline. Data associated with browsing must be stored and processed in large data centers, transmitted through global networking systems, and finally rendered on users' devices. As websites have increased in scale and technical complexity, the overall computational burden has grown substantially. Recent studies indicate that routine online activity now accounts for more than 2% of global electricity consumption [2], and this figure continues to rise as data-heavy services expand. This trend has intensified interest in identifying how energy is consumed and exploring ways to reduce usage without compromising service quality.

A significant share of website-related energy demand originates on the front end, where the browser handles layout, transitions, images, fonts, and scripts. These operations directly influence electricity usage and associated CO₂ emissions. Although developers commonly apply optimization techniques such as image compression, lazy loading, caching, and code minification, the environmental benefits of these methods are rarely evaluated. Prior research has largely centered on performance improvements, leaving the connection between front-end optimization and measurable reductions in energy consumption insufficiently examined. This study performs a series of controlled experiments to assess the effectiveness of different optimization methods in reducing energy use and emissions, contributing to a more sustainable direction for web development.

2 Background

For understanding the purpose of the study, some core concepts must be understood beforehand. The everyday web applications rely heavily on the front-end and it is the part users interact with. Front-end is composed of several core assets which include structural information (HTML), styling (CSS), interactive logic (JavaScript), images, video etc. Each of these must be transmitted over networks and processed on the end-user device. Each stage of this process uses energy. Servers and transmission networks need electricity to deliver content. Meanwhile, the client device uses more energy through its CPU, GPU, memory, and display during page rendering and interaction. So, needless to say, design choices about the front-end significantly impact on performance, user experience, energy consumption, and the resulting CO₂ emissions [19][5].

Among front-end components, images are particularly energy-intensive and thus have a great impact on the front-end, and research suggests that they account for nearly half of the average total page weight [17]. Image compression (*ic*) techniques can cut file sizes by up to 80% without sacrificing visual quality. It results in decreased bandwidth demands and also reduced energy usage on the client side to process the image, making it one of the most effective forms of optimization.

Lazy loading (*ll*) is another strategy for front-end optimization. It defers the loading of below-the-fold elements until they are required

by the user. Theoretically, it allows one to load only what is visible and does not trigger unnecessary data transfers, making the initial page render quicker. But, energy consumption can increase because of constant event monitoring causing a mixed result [16].

Code minification (*cm*) tackles one of the biggest contributors to page weight by removing comments, white space, and unnecessary characters. Minification can create space without altering functionality. Usually JavaScript alone takes on 20-30% of the total payloads [17]. So using code minification, it is possible to effectively reduce data transfer, parsing costs and increase overall device performance.

These optimizations can cut down on bandwidth usage and lower device processing needs, resulting in reduced energy consumption. So there is a strong link between front-end optimization and energy consumption. In other words, these tactics support sustainable web development. Bebinger [2] emphasizes making sustainability a design goal, yet studies on combined front-end optimization impacts remain limited. The next section addresses this problem in detail.

2.1 Problem Statement

Despite the common usage of the optimization techniques, there is very little empirical evidence on their actual effect. Many researches have highlighted the performance improvement aspect. For example, Bebinger emphasizes the need to integrate energy efficiency but also points out the lack of concrete measurements supporting their benefits [2]. Vemuri outlines strategies for lessening the carbon footprint, but lacks proper experimental evidence [19]. Ileana examines energy efficiency using distributed systems, focusing on architecture rather than client-side behavior [11]. As a result, developers still lack reliable numerical data to determine whether optimization practices meaningfully reduce energy use across devices and browsing condition. Therefore, the problem this study addresses is the lack of quantitative experimental data on front-end optimization techniques, both individually and in combination, affecting CPU energy consumption and carbon footprint.

2.2 Aim and Objectives

In order to solve the problem mentioned previously, this study explores the impact of front-end optimization on energy consumption and CO₂ emission in web applications through empirical testing under realistic usage conditions.

The objectives include:

- (1) Observe the energy consumption patterns of web applications under different front-end optimization techniques and their combinations.
- (2) Identify the key optimization techniques that contribute most to reducing energy usage during web rendering and interaction.
- (3) Estimate the overall effect of front-end optimizations on client-side energy consumption and the associated CO₂ emissions in realistic web usage scenarios.

2.3 Focus of the Work

To remain consistent with the research aim and objectives, the study focuses on the client-side perspective of energy consumption. Thus the work is limited to examining how selected front-end

optimization techniques influence energy use and CO₂ emissions on the client side of an e-commerce web application. The study considers three techniques: image compression, lazy loading, and code minification. These were applied both separately and in combination, and their effect was measured during page loading and user interaction.

The measurements only cover energy consumed at the device level, focusing on CPU usage during rendering and interaction. From these measurements, the corresponding CO₂ emissions were calculated. Measurements of server-side processes, backend architectures, and network-level energy consumption are not considered in the scope. The focus remains on the front-end, where most of the rendering and interaction occur, in order to provide clear and measurable results within defined boundaries.

3 Related Works

The digital world carries a hidden environmental cost, with websites consuming energy on each load and interaction. This section reviews 11 key works from the past decade on energy-efficient web design, focusing on image compression, lazy loading, code minification, and related strategies.”

Early work by Dick et al. [5] introduced that compression, minification, and image optimization substantially reduce data transfer, with image sizes reduced by up to 96%. Although predating modern frameworks, their findings remain influential in demonstrating that front-end techniques directly lower energy use by decreasing network and processing loads.

Building on these foundations, Rani et al. [16] proposed Energy Patterns for Web, adapting sustainability practices from mobile development. Their controlled experiments showed that lazy loading, contrary to expectations, sometimes consumed more energy than eager loading, due to the cost of scroll monitoring. This underscores the importance of empirical validation prior to implementing optimization methods.

Performance optimization has frequently been linked to energy efficiency. Vemuri [19] surveyed front-end techniques such as image optimization, lazy loading, and dead code elimination, arguing that they lower both energy use and digital carbon footprints. Similarly, Vepsäläinen et al. [20] applied script minification, CDN deployment, deferred JavaScript loading, and caching to a media-rich application, reducing page load time from 6.7 to 2 seconds. While no direct energy measurements were taken, the reductions in data transfer strongly suggest sustainability benefits.

Saarinén et al. [17] analyzed the top 10,000 websites and identified images and JavaScript as the primary contributors to page weight. By acquiring efficient image formats (WebP, AVIF) and removing unused JavaScript, payloads were reduced by up to 80%. These strategies lower both transmission costs and device energy demand.

Thallapally [18] demonstrated how async JavaScript, critical CSS, image compression, and CDN deployment improve responsiveness and indirectly reduce energy use. Ekpobimi et al. [7] extended this with a conceptual framework categorizing optimizations into resource management, rendering, and network strategies, stressing that combinations of techniques yield greater sustainability gains than isolated measures.

Bebinger [2] connected usability to sustainability, arguing that faster load times reduce device rendering duration and thus energy consumption. Platform differences are also evident: Gonçalves et al. [8] showed that browser choice affects energy consumption on Android, while Horn et al. [9] found that native Android apps consumed less energy than web-based counterparts. Dornauer and Felderer [6] provided a systematic review of a decade of mobile and web energy-saving strategies, emphasizing recurring practices such as caching, image optimization, and script minimization.

Despite these advances, benefits remain inconsistent. Some practices, like lazy loading, can even backfire depending on implementation. Crucially, no study has systematically measured the combined energy impact of multiple frontend techniques under controlled conditions. This research addresses that gap by profiling client-side CPU, GPU, and DRAM energy consumption, offering evidence-based guidance for sustainable frontend design.

3.1 Research Questions and Sub-questions

The following main research question and sub-questions were formulated to guide the study and fill the research gap addressed at the end of the previous section.

Main Research Question: What is the impact of front-end optimization techniques on the energy consumption and CO₂ emission of web applications during real-world usage scenarios?

Sub-Research Questions:

- (1) How does energy consumption vary across different front-end optimization strategies such as image compression, code minification, and lazy loading?
- (2) Which front-end components (e.g., scripts, images, CSS) contribute most to energy consumption during page rendering and interaction?
- (3) To what extent can combining multiple front-end optimization techniques reduce overall client-side energy use and the associated CO₂ emissions?

4 Methodology

Building on the research questions stated above, this study used a systematic approach to measure energy consumption and CO₂ emissions under controlled conditions. As energy use is a measurable physical process, the study used a quantitative experimental technique to collect data. Methodical observation was used to acquire knowledge, which is consistent with an experimental approach. To answer the research questions, a controlled laboratory experiment was used due to its internal validity and reproducibility. The choice aligns with methodological recommendations from Creswell (2014) [3], Denscombe (2010) [4], and Johannesson and Perjons (2014) [13], who points out that experimental control is the most valid way to draw correlations in technical researches.

Other methods, like surveys, interviews with developers, and practical trials, were considered as well. But, the required measurable energy consumption data to assess frontend improvements could not be gathered that way and it will not fulfill the aim of this study. Performing field trials could have improved ecological validity, but unpredictable differences in networks, and background activities would have lessened the value and comparison between variants would have not been possible. Furthermore, profiling at

the component level or analyzing scripts or images in isolation was not an option as it ignored the sequential and cumulative effects of different optimizations. Consequently, following the principles of small-scale experimental design, a controlled laboratory environment was employed to ensure a balance between feasibility, causal clarity, and precision (Denscombe, 2010)[4].

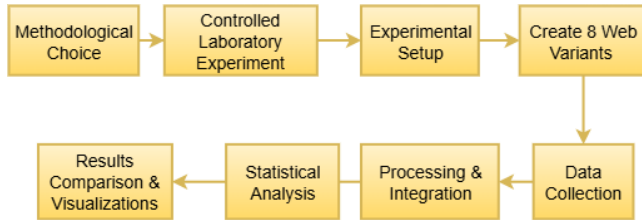


Figure 1: Methodological Workflow

According to the practical framework of this study, the consumption of energy is an observable and factual attribute of computing systems. It utilizes an empirical approach for understanding knowledge, generating insights through meticulous observation and reproducible experiments. From a methodological perspective, a controlled laboratory experiment allows for the comparison of different frontend optimization configurations and supports causal reasoning. This approach situates the investigation within the domains of computer and systems sciences, drawing upon a solid foundation grounded in sustainable software engineering and green computing research [5]. Considering all these the overall workflow of the study was designed which is given in Figure 1. Detailed discussion about the rational behind the used data collection method, is covered in the next subsection.

4.1 Data Collection Method

Energy consumption data were collected using a dual-instrument approach. The method combined data from both software and hardware sources. The Firefox Energy Profiler monitored energy consumption from the CPU and GPU, as well as estimated carbon emissions at the individual tab level, while a Tuya smart plug tracked the overall power usage of the device, including elements like the screen, storage, cooling fans, and background activities. This integrated approach allowed for comprehensive profiling along with overall system measurements, following established sustainability research techniques that stress the importance of using multiple methods to avoid reliance on a single tool [5][16][19]. The automated scripts replicated genuine user interactions such as scrolling, clicking, and idling to ensure uniformity across various configurations.

Unlike previously mentioned methods, the selected method is more effective at managing additional variables. By utilizing different optimization combinations in controlled conditions, the study provides clear quantitative data about energy consumption while remaining practical and small-scaled. This method is similar to other studies and it builds on them by specifying on numerical evidence of front-end web optimizations [18][16]. This way the study adds to previous research on green computing giving researchers a better way to compare and contrast the effects of different improvements.

This method allows an accurate and uniform assessment of energy usage and carbon emission as well as relies on previous scientific studies. But the drawbacks include limited generalization because of device configuration, possible instrument biases, and diminished ecological validity, which is common in lab settings. Even so, this method gives a useful, fact-based way for the research, leading to knowledge about energy-efficient web development and environmentally friendly software engineering.

In the next subsection the used experimental layout is described, which perfectly aligns with the chosen data collection method.

4.2 Experimental Setup

For this study the experimental subjects were eight web application variants: a baseline site, single-technique variants with image compression (*ic*), code minification (*cm*), and lazy loading (*ll*), and combined sequences such as *ic_cm*, *ll_cm*, *ll_ic*, and *ll_cm_ic*. These variants were served locally on a Windows computer using XAMPP to eliminate network variability. Sampling followed an exploratory and purposive rationale. Rather than aiming for statistical representation of all devices or networks, the study deliberately fixed the environment to one machine, one browser, and one network context, in order to isolate the causal impact of optimization sequences on energy consumption. Each variant was tested three times, leading to a planned total of twenty-four runs. Between runs, cool down intervals were integrated to minimize thermal and fan-related carryover effects.

The tools were combined into a unified system, and an automated browsing script was created to manage constant user-like activity throughout runs. As DRAM energy was not available on the test device, it was reported as 0 by the profiler. And so, the energy profiler only logged CPU package and iGPU energy. Tuya smart plug generated continuous CSV power logs. Conversely, computed aligned energy windows by using idle subtraction. Finally, the analysis scripts brought everything together. They combined the processed data, recalculated total profiler energy from its components when necessary, and worked out the average results for each site variant. Percentage changes were also measured and compared to the baseline and produced comparison figures, including bar charts. Figure 2 gives a visual outline for the whole experimental setup. In this way, the setup delivered consistent, well-timed, and checkable measurements that spoke directly to the research questions.

4.3 Data Analysis Method

To analyze the collected data using the experimental setup outlined above, the following method was used. Data from the Firefox Profiler and Tuya smart plug was processed using Python's pandas library, which ensured structured pre-processing and aggregation across measurement logs [15]. The analysis involved initial integrity checks to confirm required columns were present, with missing entries treated as zero. Profiler totals were computed by summing CPU, iGPU, and DRAM values to capture overall client-side demand. The dataset was then grouped by web variant, and the per-variant means were calculated for profiler energy, plug energy, and CO₂ emissions. The baseline variant was automatically identified, and

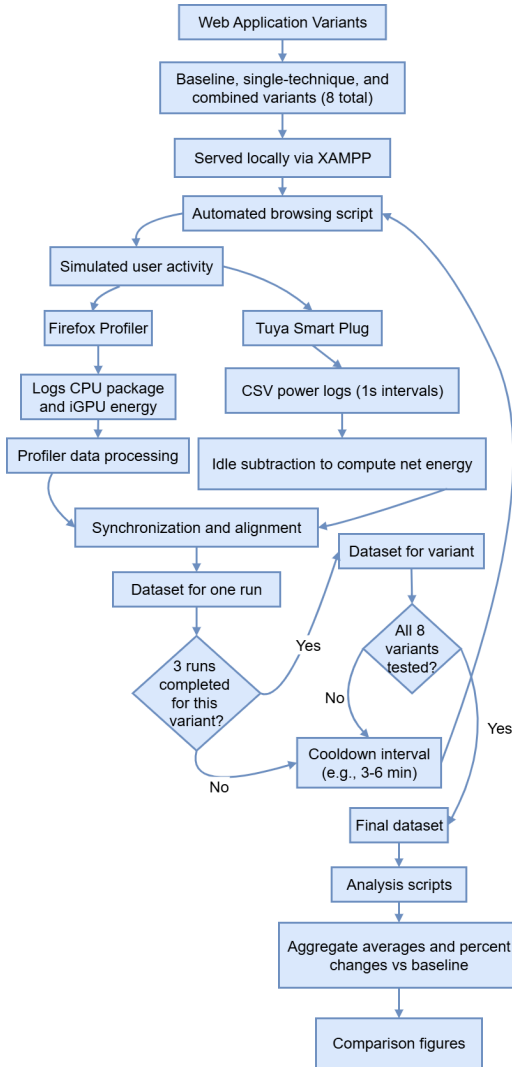


Figure 2: Experimental Setup

the percentage improvements were computed relative to this baseline.

For presentation and interpretation, descriptive statistics were exported to summary tables. Visualization was conducted using Matplotlib to generate bar charts that illustrate average consumption and improvements [10], with separate plots for each energy and CO₂ metric. All analysis was conducted in Jupyter Notebook to ensure a transparent and reproducible workflow [4]. This structured approach, including the control-based comparison which follows standard experimental practices [13], was well suited to the research questions, enabling a systematic evaluation of how front-end optimization techniques affect energy use and emissions.

The following subsection discusses the research ethics considered in this study.

4.4 Research Ethics

As the study was purely experimental, no human participants were involved, and therefore consent protocols were not applicable. All

data came from smart plug logs and system-level profiling. No identifying or personal information was collected, so GDPR-related anonymization was unnecessary. Only aggregated results were reported, while raw logs were securely stored in version-controlled directories. Scripts and procedures were recorded to enable verification of results and reproducibility. In line with established ethical guidance for digital research [14], this approach minimized risks while upholding accountability.

5 Results and Analysis

This section presents empirical findings from controlled experiments measuring energy impact of front-end optimization techniques. Results demonstrate significant variations in energy consumption across optimization strategies, providing concrete evidence of how specific front-end practices affect client-side energy use and associated CO₂ emissions.

5.1 Data Collection and Analysis Process

Data collection followed a rigorous experimental protocol designed to isolate energy impact while maintaining ecological validity. In figure 3 the overall process of data collection is shown.

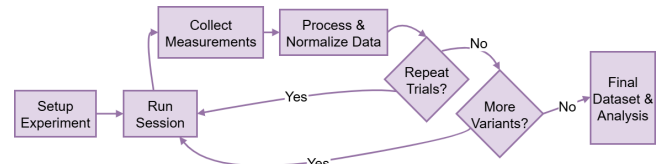


Figure 3: Data Collection Architecture

Measurements conducted on a Windows workstation using Firefox Developer Edition’s Energy Profiler for tab-level energy (CPU package and iGPU) and a Tuya smart plug for whole-device energy measurement at 1 Hz. Test website served locally to eliminate network variability.

For each of 8 variants (baseline and 7 optimization combinations), executed standardized 180-second browsing sessions using Selenium automation. Each variant tested three times (24 total runs), with 3-minute cool down periods between trials.

8-second pre-wait enabled profiler activation before consistent interaction sequence execution. CPU package energy (Wh) converted to mWh (×1000); CO₂ values (grams) converted to milligrams (×1000). Tuya plug data end-aligned to match profiler duration. Net wall energy calculated by subtracting pre-measured idle power.

Measurements were normalized to consistent units (mWh for energy, mg for CO₂). CO₂ was derived using a grid intensity factor of 442 g/kWh, following Sustainable Web Design Model methodology[1]. Plug CO₂ was computed as:

$$CO_{2,plug} (mg) = \frac{P_{plug} (mWh) \times 442}{1000}$$

Profiler tab energy was computed as CPU package + iGPU (with DRAM recorded as 0). Plug net energy was calculated as total energy minus idle power. Improvement percentages were then calculated as:

$$\text{Improvement } (\%) = \left(1 - \frac{\text{variant_mean}}{\text{baseline_mean}} \right) \times 100$$

These formulas follow standard greenhouse gas accounting practices [12, 21]. Quality control measures included pre-session idle

baseline measurements (~120 seconds), end-aligned time windows, 3-minute cool down periods, manual verification of procedural accuracy, and immediate anomaly identification through result plotting.

This structured approach, following established methodological practices in software energy measurement [3, 13], ensured reliable and replicable evidence of front-end optimizations' energy impact.

5.2 Results

This study revealed substantial differences in energy consumption across the tested optimization variants. The key findings are presented in Table 1, showing mean energy consumption and percentage improvements relative to the baseline.

The profiler energy and plug-in net energy are computed as follows:

$$E_{\text{profiler}} = E_{\text{CPU package}} + E_{\text{iGPU}}$$

$$E_{\text{plug, net}} = E_{\text{wall}} - E_{\text{idle}}$$

where $E_{\text{CPU package}}$ and E_{iGPU} are obtained using Mozilla Firefox's built-in Energy Profiler, E_{wall} is measured via a Tuya smart plug, and E_{idle} represents the baseline idle power consumption of the device under no load.

Table 1: Energy use (mWh) and CO₂ emissions (mg) across optimization variants

Variant	Energy (mWh)				CO ₂ (mg)			
	Profiler	Δ%	Plug-in Net	Δ%	Profiler	Δ%	Plug-in Net	Δ%
baseline	1557.67	0.0	1018.36	0.0	687.00	0.0	442.70	0.0
ll	1538.67	1.2	901.56	11.5	680.33	1.0	398.47	10.0
cm	1499.33	3.7	999.93	1.8	663.00	3.5	442.00	0.2
ll_cm	1134.00	27.2	127.88	87.4	501.27	27.0	56.53	87.2
ic	1019.67	34.5	535.13	47.5	461.00	32.9	236.53	46.6
ll_cm_ic	787.33	49.5	657.45	35.4	346.70	49.5	290.60	34.4
ic_cm	774.33	50.3	104.01	89.8	369.07	46.3	46.00	89.6
ll_ic	769.33	50.6	627.59	38.4	273.80	60.1	277.37	37.3

Note: Δ% denotes percentage improvement over baseline. Bold values highlight the best-performing results.

As shown in Table 1, the most effective optimization combinations produced dramatic energy reductions. When considering only the CPU and GPU components measured by the profiler (representing the tab's direct energy consumption), the **ll_ic** (lazy loading + image compression) variant achieved the **greatest reduction** at **50.6%** below baseline, followed closely by **ic_cm** (image compression + code minification) at **50.3%**. The all-three-technique combination (**ll_cm_ic**) also performed well, achieving a 49.5% reduction.

More striking results emerged from whole-device measurements (plug-in net energy). Here, the **ic_cm** combination achieved an exceptional **89.8%** reduction in energy consumption compared to the baseline, with **ll_cm** (lazy loading + code minification) following at **87.4%**. This dramatic discrepancy between profiler-based and plug-in measurements underscores the critical importance of measurement scope in energy studies.

Since CO₂ emissions were derived directly from energy consumption using a constant grid intensity factor of 442 g/kWh, the CO₂ results in Table 1 mirror the energy trends. The **ll_ic** variant achieved the greatest CO₂ reduction in profiler measurements (**60.1%**), while **ic_cm** led in plug-in measurements with an **89.6%**

reduction. This linear relationship confirms that front-end energy-saving optimizations directly translate to proportional reductions in carbon emissions.

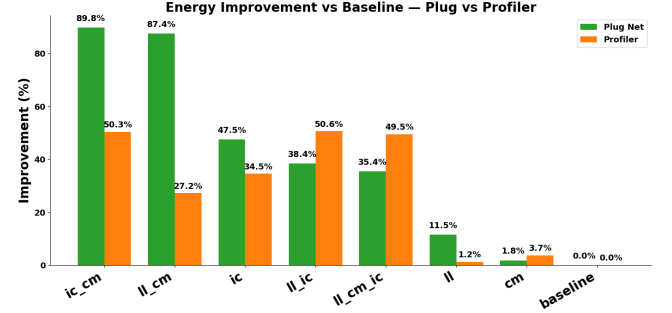


Figure 4: Energy reduction (%) for optimization variants

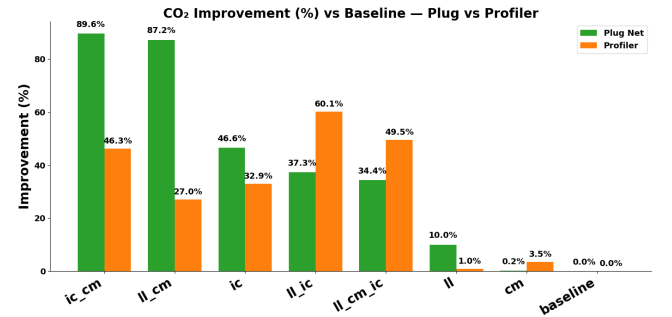


Figure 5: CO₂ reduction (%) for optimization variants

Figures 4 and 5 illustrate the percentage improvements in energy and CO₂ emissions across optimization variants. The overall trend is consistent: combined techniques consistently outperform single techniques, and image compression appears particularly impactful when combined with other optimizations.

6 Discussion

This section discusses the study's findings in three parts: the answers to the research questions, the significance of the results, and the comparison with the existing literature. Together, these subsections provide a comprehensive interpretation of the experimental outcomes and their broader implications.

6.1 Answers to the Research Questions

Front-end optimization techniques substantially reduce energy consumption and CO₂ emissions. The most effective combinations achieved **49.5–50.6% component-level** and **87.4–89.8% system-level** reductions versus *baseline*. These directly translate into equivalent CO₂ reductions (**49.5–60.1%** and **87.2–89.6%**), confirming the linear energy–emissions relationship. These findings answer the **main research question**. For **SubRQ1**, individual effectiveness varied. *ic* delivered the strongest results (**34.5% component-level**, **47.5% system-level**). *cm* achieved modest gains (**3.7%**, **1.8%**), while *ll* showed minimal component impact (**1.2%**) but larger system effect (**11.5%**).

For **SubRQ2**, component analysis revealed distinct patterns. Image handling strongly influenced iGPU energy, with *ic* lowering

iGPU use by $\approx 40\%$. JavaScript processing mainly impacted CPU, with *cm* reducing CPU package energy by 5–7%. Combinations such as *ic_cm* and *ll_ic* reduced both CPU and iGPU energy, confirming both components substantially drive consumption.

For **SubRQ3**, combinations outperformed single methods. While individual techniques reached at most **34.5% reduction**, combinations achieved about **50%**. System-level results were dramatic, with *ic_cm* reaching **89.8% reduction**. Order mattered: *ic_cm* outperformed *cm_ic*. Three-technique sets gave diminishing returns, e.g., *ll_cm_ic* produced **49.5% component-level** but only **35.4% system-level** reduction.

6.2 Significance of the Results

These findings have theoretical, practical, and societal value. The main contribution of the study is to experimentally determine how image compression, lazy loading, and code minification affect client-side energy use and CO₂ emissions. The measurements confirm that image compression provided the strongest single benefit, reducing system-level energy use by 47.5%, and that the combination of image compression and code minification reduced energy and emissions by almost 90%. These contributions place the study within current research on sustainable web development. They provide quantitative evidence of how common front-end techniques change energy behavior.

In practical terms, effective combinations, such as *ic_cm* and *ll_ic*, reduce consumption by about 50% at the component level and up to 90% at the system level. *ic* proved to be the most beneficial single technique, making it a key focus for sustainability. These experimental results support the idea that front-end techniques can significantly change the energy use of web applications. Including multiple optimization sequences also adds to the depth of analysis in the study. The study demonstrates several strengths that support the reliability and interpretation of the results. It also addresses the growing concern surrounding the carbon footprint of the web and provides empirical measurement-based evidence for sustainable front-end practices. The controlled laboratory environment, automated browsing sequences, dual-instrument energy measurement, and repeated trials allow consistent data collection and clear comparison across variants. Integrating both component-level measurements, such as CPU and iGPU, and whole-device measurements helped to understand how specific optimization strategies affect energy use. The study presents clear data showing reductions of up to 90%. It also offers practical advice on which optimization sequences yield the highest environmental benefits.

Societally, these findings show that digital technologies already use over 2% of global energy. The tested techniques indicate that can be achieved by measurable reductions in CO₂ emissions by using standard front-end practices. The multi-level measurement approach used in the study gives a clear picture of device-level behavior and helps in a better understanding of how specific design choices affect environmental outcomes.

6.3 Comparison with Existing Literature

Table 2 systematically compares the findings of this experiment against key literature, demonstrating how this study confirms, refines, and extends prior research on sustainable web development

through empirical validation of optimization techniques' energy impacts.

This comparative analysis reveals several critical insights about the relationship between the findings and the existing literature. First, the work provides empirical validation for theoretical claims made in foundational studies like Dick et al.[5], which proposed Green Web Engineering principles but lacked direct energy measurements. By quantifying that image compression alone reduces energy consumption by 34.5–47.5%, the study provides the concrete evidence that was previously missing.

Second, the study extends performance-oriented research [18] by demonstrating that the relationship between performance improvements and energy savings is not always linear. While such studies documented load time reductions from front-end optimizations, they did not investigate how these performance gains translate to energy consumption. The findings reveal that while performance improvements generally correlate with energy savings, the magnitude varies significantly by optimization technique and measurement scope, highlighting a crucial nuance for sustainable web development.

Third, the findings refine the understanding of lazy loading's energy impact. While Rani et al.[16] reported cases where lazy loading increased energy due to scroll monitoring overhead, the measurements show that its effectiveness is highly context-dependent: when used alone, lazy loading provides minimal energy benefits (1.2% reduction), but when combined with image compression (*ll_ic*), it becomes one of the most effective strategies (50.6% reduction). This contextual understanding represents a significant advancement in the field.

Moreover, the results align with the work of Saarinen et al.[17], who emphasized the role of payload reduction in sustainability. The study confirms that image compression and code minification both decrease CPU and iGPU load, but the contribution advances this by quantifying the extent of reductions across different measurement scopes. Similarly, the long-term survey by Dornauer and Felderer [6] identified caching, compression, and minimization as recurring effective strategies; the work builds on this by showing that not only do these techniques work, but their specific combinations and sequences determine the scale of energy savings.

Finally, by situating the findings alongside the focus of carbon-footprint Vemuri [19], prior claims are extended by providing synchronized profiler- and plug-level measurements. This dual-scope methodology demonstrates that front-end optimizations can reduce whole-device energy by up to 89.8%, thereby offering developers practical, evidence-based guidance on how to prioritize optimization techniques for maximum sustainability benefit.

7 Limitations and Future Work

The study has several limitations that can affect the interpretation of its findings. All measurements took place on a single Windows workstation using the Firefox browser, which limits the relevance of the results to other devices, operating systems, and browsers. Running the website locally removed network delays, bandwidth fluctuations, and CDN-related factors. As a result, the findings primarily reflect device-level behavior rather than end-to-end delivery conditions. The energy use of DRAM was not measurable on the

Table 2: Comparison of findings with related literature

Paper	Contribution	Findings
Dick, Naumann & Held (2010) [5]	Green Web Engineering principles; image optimization and streamlined code	Front-end optimizations reduce data transfer and processing, lowering energy demand
Vemuri (2021) [19]	Front-end optimization (minification, compression, caching)	Reported general improvements but lacked direct measurement of device-level energy
Rani et al. (2024) [16]	Energy patterns for web applications	Lazy loading can reduce network activity but may have mixed energy impact
Saarenin et al. (2025) [17]	Investigated webpage bloat and data-heavy design	Reducing payload size is critical for sustainability
Thallapally (2023) [18]	Best practices for front-end performance	Efficiency benefits focus on speed/load times
Dornauer & Felderer (2023) [6]	Survey of mobile web energy-saving strategies	Caching, compression, and resource minimization are effective
Contribution of this study	Confirmed (Dick et al., 2010): Image compression most effective (35–47% reduction). Extended (Vemuri, 2021): Quantified profiler and wall-socket energy; CO ₂ reduction up to 90%. Partially confirmed (Rani et al., 2024): Lazy loading modest alone, effective in combination (up to 50.6% profiler, 87.4% plug). Confirmed (Saarenin et al., 2025): CPU/IGPU usage reduction up to 50.6%. Diverged (Thallapally, 2023): Not all performance gains translate into energy savings; sequence effects matter. Confirmed (Dornauer & Felderer, 2023): Combination analysis reveals sequence-dependent effects.	

Notes: LL = lazy loading, IC = image compression, CM = code minification.

test device and therefore, was recorded as zero, leaving this component of client-side activities unexplored. Additionally, focusing on a single e-commerce site collection limits the generalizability of the findings, as different site architectures may exhibit varying rendering and interaction characteristics.

The analysis examined only three optimization strategies and provided narrative descriptions of the results without conducting inferential statistical analysis. This makes it challenging to determine the statistical significance of differences observed across repeated tests. The research concentrated on energy-related metrics and did not evaluate usability impacts, such as potential reductions in visual quality resulting from image compression or interaction effects associated with lazy loading. The study also did not compare optimization behavior across browsers or analyze internal caching, decoding processes, or rendering pipelines used by alternative browser engines.

Future work could involve testing on a wider range of devices with different hardware configurations, operating systems, and browsers to improve the generalizability of the results. Investigating additional optimization techniques—such as converting images to modern formats like WebP or AVIF, bundling and minifying scripts, leveraging caching strategies, and incorporating CDNs—may provide a more comprehensive understanding of sustainable web performance practices. More precise power measurements that include memory, disk activity, and real network conditions could yield a fuller view of client-side energy behavior. Exploring different categories of websites and incorporating usability and perceptual evaluations would help clarify the trade-offs between sustainability and user experience. Finally, conducting statistical analyses

across repeated trials could improve interpretability and strengthen comparisons among optimization techniques.

8 Conclusion

This study systematically evaluated how front-end optimization techniques impact energy consumption and CO₂ emissions in web applications. Through controlled measurements of eight website variants, the study demonstrated that image compression, lazy loading, and code minification reduce client-side energy use, with combined techniques achieving substantial savings under the test conditions.

On tab-scoped profiler metrics, the strongest sequences reduced CPU and GPU energy by roughly half. At the wall, net-above-idle reductions were substantially larger when optimized variants kept the system near idle during the measured window. The ordering of techniques affects whether savings appear primarily in tab-scoped tracks or in whole-device draw. Practitioners should report both instrument scopes and document timing and idle procedures to allow correct interpretation and replication.

The primary contribution is quantified evidence of combined optimization effects under synchronized profiler and wall measurements. Future work should aim at standardized methods that capture both component-level and whole-device energy for consistent cross-study comparisons.

Simple, widely used front-end practices such as image compression, code minification, and lazy loading are not just performance enhancers but measurable energy-saving strategies. When applied thoughtfully and in the right sequence, these techniques can cut system-level energy use by up to 90%, making them an immediate and practical pathway toward lowering the environmental footprint of everyday web applications.

References

- [1] [n. d.]. Estimating digital emissions. <https://sustainablewebdesign.org/estimating-digital-emissions/>. Accessed: 2025-09-05.
- [2] Hannah Bebinger. 2022. Sustainability in Web Development through Energy Efficiency. Honors Project, Bowling Green State University, Spring 2022. <https://scholarworks.bgsu.edu/honorsprojects/790/>
- [3] John W. Creswell. 2014. *Research Design: Qualitative, Quantitative, and Mixed Methods Approaches* (4 ed.). SAGE Publications, Thousand Oaks, CA.
- [4] Martyn Denscombe. 2010. *The good research guide: For small-scale social research projects*. McGraw-Hill Education, Maidenhead, UK.
- [5] Markus Dick, Stefan Naumann, and Alexandra Held. 2010. Green web engineering: A set of principles to support the development and operation of “green” websites and their utilization during a website’s life cycle. In *Proceedings of the 6th International Conference on Web Information Systems and Technology (WEBIST ’10)*. SciTePress, Valencia, Spain, 48–55. <https://doi.org/10.5220/0002805200480055>
- [6] Benedikt Dornauer and Michael Felderer. 2023. Energy-saving strategies for mobile web apps and their measurement: Results from a decade of research. In *Proceedings of the 2023 IEEE/ACM 10th International Conference on Mobile Software Engineering and Systems (MOBILESoft)*. IEEE, Melbourne, Australia, 75–86.
- [7] Harrison Oke Ekpobimi, Regina Coelis Kandekere, and Adebamigbe Alex Fasanmade. 2024. Conceptual framework for enhancing front-end web performance: Strategies and best practices. *Global Journal of Advanced Research and Reviews* 2, 1 (2024), 99–107.
- [8] Nélson Gonçalves, Rui Rua, Jácome Cunha, Rui Pereira, and João Saraiva. 2022. Energy efficiency of web browsers in the Android ecosystem. *arXiv preprint arXiv:2205.11399* (2022).
- [9] Ruben Horn, Abdellah Lahnaoui, Edgardo Reinoso, Sicheng Peng, Vadim Isakov, Tanjina Islam, and Ivano Malavolta. 2023. Native vs web apps: Comparing the energy consumption and performance of Android apps and their web counterparts. In *Proceedings of the 2023 IEEE/ACM 10th International Conference on Mobile Software Engineering and Systems (MOBILESoft)*. IEEE, Melbourne, Australia, 44–54.

- [10] John D. Hunter. 2007. Matplotlib: A 2D graphics environment. *Computing in Science & Engineering* 9, 3 (2007), 90–95.
- [11] Marian Ileana. 2023. Optimizing energy efficiency in distributed web systems. In *Proceedings of the 2023 7th International Symposium on Innovative Approaches in Smart Technologies (ISAS)*. IEEE, 1–5. <https://doi.org/10.1109/ISAS60782.2023.10391617>
- [12] International Energy Agency. 2025. World Energy Outlook 2025. <https://www.iea.org/reports/world-energy-outlook-2025> Accessed: 2025-09-14.
- [13] Paul Johannesson and Erik Perjons. 2014. *An Introduction to Design Science*. Springer, Cham, Switzerland.
- [14] Annette Markham and Elizabeth Buchanan. 2012. Ethical decision-making and internet research: Recommendations from the AoIR ethics working committee. <https://aoir.org/reports/ethics2.pdf>. Version 2.0.
- [15] Wes McKinney. 2010. Data structures for statistical computing in Python. In *Proceedings of the 9th Python in Science Conference*. 51–56.
- [16] Pooja Rani, Jonas Zellweger, Veronika Kousadianos, Luis Cruz, Timo Kehrler, and Alberto Bacchelli. 2024. Energy patterns for web: An exploratory study. In *Proceedings of the 46th International Conference on Software Engineering: Software Engineering in Society (ICSE-SEIS '24)*. Association for Computing Machinery, Lisbon, Portugal, 12–22. <https://doi.org/10.1145/3639475.3640110> Supplementary material available at <https://doi.org/10.5281/zenodo.8404487>.
- [17] Aleksi Saarinen, Muhammad Farooqi, Matti Pärssinen, and Jukka Manner. 2025. Understanding and mitigating webpage data bloat: Causes and preventive measures. *ACM SIGEnergy Energy Informatics Review* 4 (2025), 106–112. <https://doi.org/10.1145/3727200.3727216>
- [18] Nagaraju Thallapally. 2023. Best practices for enhancing front-end performance in modern web development. *Journal of Computer Science and Technology Studies* 5, 2 (2023), 47–53. <https://doi.org/10.32996/jcsts.2023.5.2.4>
- [19] Venkata Padma Kumar Vemuri. 2021. Greening the web: Front-end optimization for reducing energy consumption and carbon footprints. *International Journal on Science and Technology (IJST)* 12, 1 (2021), 1–10.
- [20] Juho Vepsäläinen, Arto Hellas, and Petri Vuorimaa. 2024. Overview of web application performance optimization techniques. arXiv:2412.07892 [cs.SE] <https://arxiv.org/abs/2412.07892>
- [21] World Resources Institute and World Business Council for Sustainable Development. 2004. *The Greenhouse Gas Protocol: A Corporate Accounting and Reporting Standard (Revised Edition)*. World Resources Institute, Washington, DC. <https://ghgprotocol.org/corporate-standard>

Appendix A

This study did not involve human participants, therefore no informed consent was required.

Appendix B

Data Collection Protocols

The data collection protocols are described in the Data Collection and Analysis Process subsection of this report and are not repeated here.

Survey/Interview Instruments

Not Applicable as this study did not involve surveys or interviews.

Questionnaire

Not Applicable as no questionnaire was used in this study.

Supplementary Content

The following supplementary materials were used in the study. Raw energy measurement logs collected using the Tuya smart plug and profiler is provided inside GitHub repository. Some screenshots of web pages Fig 6 Fig 7 Fig 8 are provided below.

Appendix C

Dataset and Project Repository

All datasets, experimental code, and supplementary materials are available at the following GitHub repository:

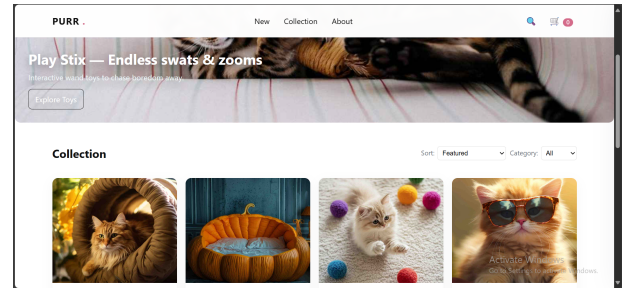


Figure 6: HomePage

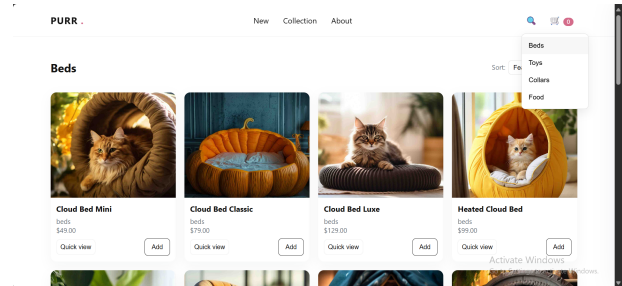


Figure 7: Categories

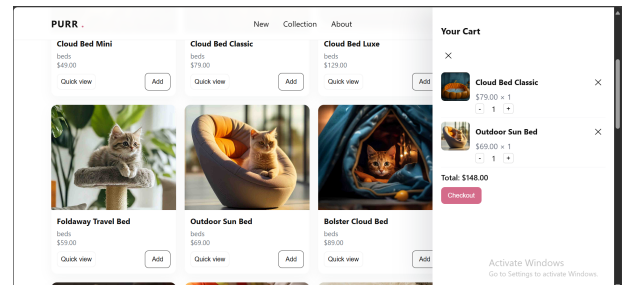


Figure 8: Cart

<https://github.com/Jerinan27/Energy-Efficient-Web-Design-Measuring-Impact-of-Front-end-Optimizations>

The repository includes: Raw energy consumption data (CSV format), scripts for data collection and analysis, sample optimized and baseline website files. Documentation for replicating the experiments.